capture groups are set, or if a lookaround was a success. After the conditional part of the pattern, if the condition is met (in other words, evaluates to true) then another expression is then tested against the sample for a match. Optionally, an alternative expression can be applied if the condition is not met. The general form of the syntax is `(?(IF)THEN|ELSE)`. As ever, here's an example to demonstrate:

```
preg_match_all('/(USD)?\s(?(1)(\d+))/', 'USD 150', $matches);
print_r($matches);
```

```
Output:
    [0] => Array
        (
            [0] => USD 150
        )

    [1] => Array
        (
            [0] => USD
        )

    [2] => Array
        (
            [0] => 150
        )
```

Let's look at the regex itself – `/(USD)?\s(?(1)(\d+))/` – and break it down. The portion before the conditional sub-pattern is `(USD)?\s` which matches the literal string `'USD'` inside a capture group, with a following `\s` space character type. The quantifier `?` means "zero or one times," and its position is very important. If it was inside the capture group, then this would have the effect of always setting the back-reference (either containing the `'USD'` string, or nothing), but by placing it outside the capture, it can happen zero times, which means the back-reference is only created on a match with `'USD'`. This is crucial for the next part of the pattern: `(?(1)(\d+))`, which tests for the existence of the back-reference with `(1)`; if it exists, then the next part of the sub-pattern is applied, which is `(\d+)`, and captures all digit characters after the space. The output shows the full pattern match (the entire sample string in this case), the first capture group contents `'USD'`, and the second capture group contents, which are the digits `'150'`.